

Paging, Virtual Memory, and Swap

What we ran in class, explained so you can run it again yourself

This walks through the commands we ran live in class, in order, with an explanation of what each one is doing and why. Everything here works on your own machine exactly as it worked on the demo machine, since it has been checked against Arch, Void, Gentoo, Artix, and Xubuntu. Nothing below needs root, since every step only ever reads information about your own processes or about the system in general, and nothing here is destructive, so do not hesitate to rerun it, change a value, and see what happens.

A note on running this yourself. Every command below relies only on Python 3 and standard utilities that already ship with each of the five distributions in the room. There is nothing extra to install.

1. Day 2: Watching Paging, Virtual Memory, and Swap

This is the demonstration for the day we cover partitioning, paging, virtual memory, and swap. Read it alongside Figure 9.16, logical and physical addresses, and the paging and virtual memory material that follows it.

Step 1. Look at a process's virtual address space

```
sleep 300 &  
PID1=$!  
sleep 300 &  
PID2=$!  
cat /proc/$PID1/maps
```

`$!` holds the process ID of whatever you most recently sent to the background. Unlike the job numbers we used on Day 1, which only worked with `kill`, reading a file under `/proc` needs the actual numeric process ID, which is exactly what `$!` gives you, so we save it into a variable right away before starting the next background job overwrites it.

The output of `cat /proc/$PID1/maps` is a list of the address ranges this one process can see, each with its permissions (read, write, execute), and, where it applies, which file on disk that range is backed by. On a real desktop process, not the bare `sleep` used here, you will typically see a handful of distinct regions: the program's own code and data, one entry for each shared library it has loaded, a heap, and a stack. Every one of these is a separate mapping into the process's own private view of memory, which is exactly what Figure 9.16 is describing when it draws a logical address space that is translated into physical addresses somewhere else entirely.

Now compare the two processes directly.

```
cat /proc/$PID2/maps
```

The two sets of address ranges will look remarkably similar, often nearly identical, even though these are two completely separate processes with completely separate physical memory behind them. That similarity is virtual memory working exactly as intended: every process is handed what looks like its own private address space, starting from familiar looking numbers, and it is entirely the OS's job, with hardware help, to translate those into whatever physical memory each process actually has.

Step 2. Every mapped region is a whole number of pages

```
getconf PAGE_SIZE
```

This reports the size, in bytes, of a single page on this machine, almost certainly 4096. Pick any range from Step 1's output, written as two hexadecimal addresses separated by a dash, and subtract the first from the second. The result will always come out to a whole multiple of the page size. This is not a coincidence: physical memory is only ever handed out to a process one whole page at a time, never in smaller pieces, which is the entire premise behind paging as a way of managing memory.

Step 3. Watch a page get assigned only when it is touched

Save the following as `demandpaging.py`.

```
cat > demandpaging.py << 'EOF'
import os, mmap, time

pid = os.getpid()
size = 200 * 1024 * 1024
print(f"PID {pid}: reserving {size // (1024*1024)} MB, untouched")
m = mmap.mmap(-1, size)
print("reserved. check VmSize and VmRSS now, in the other window")
time.sleep(20)

touch = 40 * 1024 * 1024
print(f"touching the first {touch // (1024*1024)} MB")
m[0:touch] = b"x" * touch
print("touched. check VmSize and VmRSS again")
time.sleep(60)
EOF
```

Then run it and watch it from the same terminal, since it prints its own progress while it runs in the background.

```
python3 demandpaging.py &
PID=$!
grep -E "VmSize|VmRSS" /proc/$PID/status
```

Run that `grep` again a few seconds later, still inside the twenty second window the script is waiting through, and you should see `VmSize` already sitting at roughly 200 MB while `VmRSS` has barely moved from wherever it started. This is the divergence promised on Day 1: `VmSize` counts everything this

process has reserved as address space, the moment it reserves it, while **VmRSS** only counts pages that have actually been assigned physical memory. Reserving two hundred megabytes with **mmap** does not touch a single physical page. It only tells the kernel to hold that range of addresses for this process and to fill in real memory later, lazily, the first time each part of it is actually read or written.

Once the script prints that it has touched the first forty megabytes, run the same **grep** again.

```
grep -E "VmSize|VmRSS" /proc/$PID/status
```

VmRSS should have grown by roughly forty megabytes, while **VmSize** has not moved at all, since nothing new was reserved, only written to. That growth is demand paging happening in real time: the instant this process wrote to a byte inside a page it had reserved but never touched, the processor raised a fault, the kernel caught it, handed over one real physical page, and only then let the write actually complete. Every one of those forty megabytes of newly resident memory arrived as a sequence of individual page faults, one page at a time, not as a single bulk allocation.

Step 4. Check what swap looks like on this machine

```
free -h  
swapon --show  
cat /proc/swaps
```

free -h summarizes physical memory and swap together: how much RAM exists, how much is in active use, and how much swap space exists and is currently occupied. **swapon -show** and **/proc/swaps** both list the specific swap devices or files configured on this machine, if any. It is entirely possible, and increasingly common on machines with a lot of RAM or a fast SSD, for this to come back showing no swap configured at all. That is not a broken result. It is worth treating as its own discussion point: swapping only helps once physical memory is genuinely under pressure, and a machine that rarely runs low on memory may not be configured with any swap to fall back on in the first place.

Notice that a process stalled on a page fault, waiting for the kernel to hand it a physical page, is doing exactly what Day 1 called the Waiting state: suspended until a resource it needs becomes available. Paging did not introduce a new process state. It only gave the OS one more reason to move a process into a state you already know.